Eliminating Timing Channels in Microkernels

Integrating Time Protection Mechanisms into FreeRTOS

Simon V. Brodersen, 16–06–2026

Timing Channels: The Problem

- Timing channels leak information through observable execution time
- Real-world attacks:
  - Cryptographic key extraction (Meltdown, Spectre)
  - Cross-VM information leakage in cloud computing
- The OS scheduler itself is a channel:
  - Priority-based schedulers leak task runnability
  - Shared caches and branch predictors leak state
- **Core challenge:** Closing timing channels at the OS level

Research Question & Contributions

Research Question:

Can time protection mechanisms be integrated into a mainstream OS to mitigate timing channels?

Contributions:

1. A **domain-level scheduler** partitioning execution into fixed time slices
2. **Temporal isolation** via the `fence.t` instruction on domain switches
3. **Evaluation** on QEMU showing constant-time domain scheduling

Background: Timing Channels

Three categories of timing channels in operating systems:

- **Scheduler-based:** Priority decisions leak runnability info
- **Shared-state:** Caches, TLBs, branch predictors retain cross-task state
- **Delay-based:** Interrupts and locks delay execution predictably

Isolation as the countermeasure:

- *Spatial:* Memory partitioning (MMU / MPU)
- *Temporal:* Ensuring shared resource usage in one domain does not affect another

The `fence.t` Instruction

- Hardware-assisted temporal isolation for RISC-V
- Flushes on-core microarchitectural state on domain switches:
  - L1 caches, TLBs, branch predictors
  - LFSR state, memory arbiters (full flush)
- Overhead in seL4: only **0.7%**
- Software-supported variant (`fence.t.s`) for out-of-order cores: **1.0%**

User-level mitigations

By easing the restriction of time protection to allow for non-complete partitioning, we give way to some user-level implementations that help reduce timing channels.

- **An example:** FaCT is a domain specific language, which sought to make it easier to write cryptographic code
- The programmer marks secret information, and FaCT modifies the source code such that it become constant-time in respect to the secret information
- Seemingly safe code at the C-level

Shortcomings:

- Variable execution time of operators, for example the numerator of division
- Compiler introduced optimizations
- Is scheduling sensitive information?

The Missing OS Abstraction

The missing security components not addressed with user-level implementations gave way to more custom OS implementations.

sel4 Time Protection: Introduced the notion of time protection at the OS-level.

- Build as a modification to the existing sel4 kernel
- Copies the kernel code into each security domain
- Flushes shared hardware, which can not be spatially partitioned
- Cache-coloring to partition higher level caches

Qian Ge et al. "Time Protection: The Missing OS Abstraction". In: *Proceedings of the Fourteenth EuroSys Conference 2019*. EuroSys '19. New York, NY, USA: Association for Computing Machinery, Mar. 25, 2019, pp. 1–17. ISBN: 978-1-4503-6281-8. DOI: 10.1145/3302424.3303976. URL: https://dl.acm.org/doi/10.1145/3302424.3303976 (visited on 02/03/2026)

S3K capability-based RTOS

Another approach to Time Protection at the OS-level comes from S3K. S3K aimed to provide a more dynamic implementation.

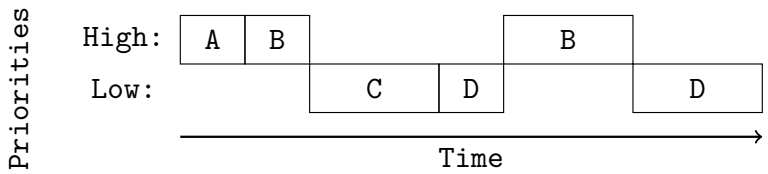
- A capability based model, which extends to time slices
- A time-driven, capability-based scheduler with systematic use of `fence.t`
- Predictable, constant-time system calls
- Scratchpad memory for kernel code execution (backed by L1 cache, flushed on scheduling)

The Gap: No prior work integrates these mechanisms into an *existing* mainstream OS like FreeRTOS.

Henrik Karlsson and Roberto Guanciale. "Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning". In: *2025 IEEE Real-Time Systems Symposium (RTSS)*. 2025 IEEE Real-Time Systems Symposium (RTSS). Dec. 2025, pp. 68–81. DOI: 10.1109/RTSS66672.2025.00015. URL: https://ieeexplore.ieee.org/document/11315078 (visited on 02/04/2026)

Vulnerability: Priority-Based Scheduling

FreeRTOS uses a priority-based preemptive scheduler, which is inherently unfair and vulnerable.



The channel: A high-priority task signals 0 or 1 by sleeping or running, and the low-priority task measures its own wake time.

Chien-Ying Chen et al. "A Novel Side-Channel in Real-Time Schedulers". In: *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). Apr. 2019, pp. 90–102. DOI: 10.1109/RTAS.2019.00016. URL: https://ieeexplore.ieee.org/document/8743170/ (visited on 02/09/2026)

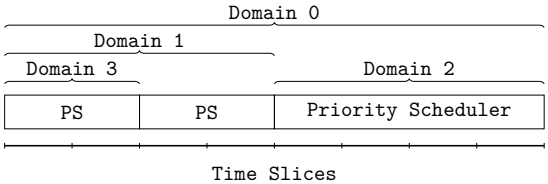
Possible Solutions

To solve the priority based scheduling, a few different approach were considered.

- **Fixed execution time:** Each priority could have a known fix execution time and scheduling.
- **Introduction of Domains:**
  - A domain is a group of tasks, which the scheduler treats as a single environment
  - Only partition across domain boundaries, allows for intra-domain channels
  - Requires the notion of time slices, which is a fixed execution time serving as the smallest unit of "time" for the scheduler
  - Should domains be dynamic or static?

Design: Domain Scheduling

Chosen approach: Domain scheduling



- Initially all time slices are given to Domain 0
- Domain 0 gives up fixed time slices to other domains on creation
- Tasks within a domain use standard priority scheduling
- Allows for a trade-off between the throughput (priority) and partitioning (domain)

Implementation: Extending the FreeRTOS API

FreeRTOS MPU allows for the creation of restricted tasks through the `xTaskCreateRestricted`. To introduced the notion of domains, the API for creating tasks are extended with the following parameter.

```
typedef struct xDOMAIN_PARAMETERS {
  uint32_t ulSliceIndex;   // Start time slice
  uint32_t ulSliceLength;  // Number of slices
} DomainParameters_t;
```

- When creating a restricted task, one must pass the start of the group of time slices and their length
- If no `DomainParamters_t` is passed, then it creates the tasks within the current domain

Implementation: Domain Switching

The general time unit within FreeRTOS is a *Tick*. In the RISC-V version of FreeRTOS the Tick is incremented when there is a machine timer interrupt.

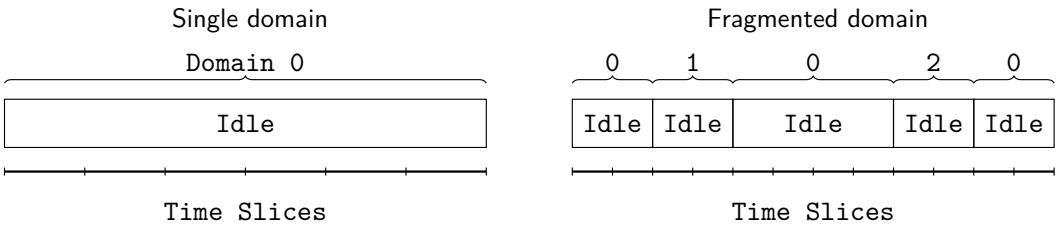
Idea: Hook into `xTaskIncrementTick` to check for domain switches:

```
xDomainTick = (xEffectiveTick / NUM_TICKS_PER_SLICE)
                % NUM_TIME_SLICES;

if (xDomainTick >= current + length || xDomainTick < current) {
    ...
    temporal_fence_t();          // Flush microarchitectural state
    pxCurrentDomainIndex += length;
    // ... restore next domain's task
}
```

- Keep track of our own xDomainTick, to check if we should schedule a new domain
- Flush when a domain switch is needed

Implementation: Constant-Time Domain Creation



- `xDomains` array indexed by time-slice start
- `uxNext` linked list finds parent blocks in **constant time**
- Index arithmetic splits left / right fragments without scanning

Implementation: Cross-Domain Queues

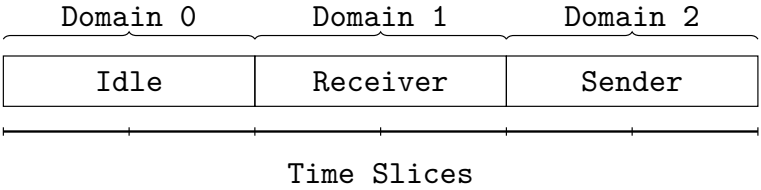
In FreeRTOS `Queues` are used as the primary tool for communication between different tasks.

- There exists a ready queue, which tracks tasks that are ready to be executed
- When a task requests a message on the `Queue`, it adds itself to a list of **waiting** tasks
- When the `xQueueSend` sends a message on the `Queue`, it moves all the waiting tasks to the ready queue. Tasks waiting on the `Queue`, might now come from different domains
- **Solution:** Track which domain each task belongs to, to move them correctly.

This introduces clear timing channels between domains, but they are restricted to tasks on the `Queue`.

Evaluation: Blinky Demo

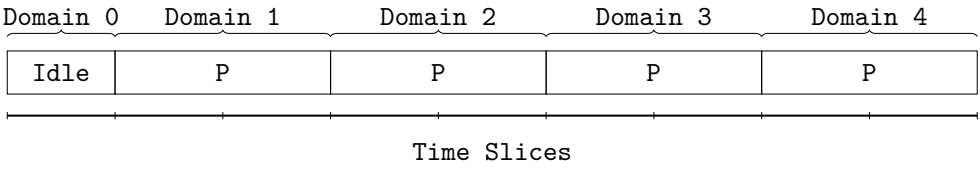
Two tasks in separate domains communicating via a Queue:



- Sender scheduled at domain tick 4, Receiver at domain tick 2
- Round-trip time measured with `rdcycle`: **50,000 cycles ± 1 cycle**
- Demonstrates constant-time domain switching

Evaluation: Multi-Domain Demo

4 domains, each with a privileged task that creates a higher-priority unprivileged task:



- Domain ticks are deterministic (1, 3, 5, 7, ...)
- Priority scheduler still works within each domain
- Trade-off: domain-relative delays, added complexity, reduced throughput

Conclusion

What I showed:

- Time protection mechanisms *can* be integrated into an existing mainstream OS
- Domain-level scheduler achieves constant-time scheduling in QEMU
- Trade-offs
  - **Complexity:** A significant amount of complexity has been added to the code. Future work onFreeRTOS would be more cumbersome with the extra code complexity.
  - **Performance:** Given the implementation makes use of idle time, the total performance is most likely reduced significantly. Further, more memory accesses is needed for the domain Scheduler to function, so while asymptotically the same, it is a larger overhead.

Conclusion: Full domain isolation is possible and functional in general operating systems.

Future work

- Evaluation on real hardware with `fence.t` support (or gem5)
- Review system calls: critical sections delay scheduling, only user-level protection
- Introduce a two level locking system.
  - FreeRTOS has a notion of "critical section". These sections grab a global lock to ensure consistency of the priority-based scheduler.
  - Currently the domain level scheduler uses the same lock.
  - **Idea:** Allow for a separate locking mechanism for cross-domain consistency. This would allow scheduling a new domain while holding the priority-based scheduler lock.
- Remaining channels: interrupts, higher-level caches

Takeaway

While a full domain level scheduling setup does function, it performs worse and adds complexity. The primary goal of the general OS is throughput, the domain level scheduler seems overkill. An idea could be to introduce aspects.

- Allow for the creation of "secure" tasks, which would be partitioned. Two general aspects could be tested
  - Flush before and after a secure task, and allow for partitioned higher level caches.
  - Introduce the Scratchpad Memory. Here one could either restrict the task to this memory footprint, or use the Scratchpad Memory as a buffer, which we can index with secrets. This would require loading all data into the SPM.